

```
In [ ]: import marimo as mo
```

Trabalho Prático 1.1: Gerador de Horário Escolar

Catarina Rodrigues (a111491)

Francisca Cardoso (a112105)

1. Introdução e Modelação do Problema

Este projeto visa a construção de um horário escolar otimizado, respeitando um conjunto rigoroso de restrições de recursos (turmas, professores, salas e tempos letivos) e reagindo dinamicamente a alterações (construção incremental). Optou-se pela utilização do solver CP-SAT da biblioteca OR-Tools por se tratar de uma ferramenta moderna de Programação por Restrições (Constraint Programming), altamente eficiente na gestão de restrições lógicas complexas, variáveis booleanas e inteiras, superando as abordagens tradicionais de Programação Linear Inteira (MIP) em problemas combinatórios de horários escolares.

```
In [ ]: import pandas as pd
        from ortools.sat.python import cp_model

turmas_df = pd.read_csv("dados/turmas.csv")
disciplinas_df = pd.read_csv("dados/disciplinas.csv")
salas_df = pd.read_csv("dados/salas.csv")
excecoes_df = pd.read_csv("dados/disponibilidade_excecoes.csv")

turmas_df
```

2. Definição do Modelo Matemático

Começamos por definir os conjuntos do domínio do problema:

- T : Conjunto de Turmas
- D : Dias da semana (Seg a Sex)
- Per : Períodos letivos diários (1 a 5)
- S : Conjunto de Salas disponíveis
- $Prof$: Conjunto de Professores
- $Disc$: Conjunto de Disciplinas do currículo

Variáveis de Decisão (X)

O modelo assenta num dicionário de variáveis booleanas. Seja a variável:

$$X_{t,disc,p,d,per,s} \in \{0, 1\}$$

Assume o valor 1 se a turma t tem aula da disciplina $disc$, com o professor p , no dia d , período per , na sala s . Assume 0 caso contrário.

```
In [ ]: TURMAS = turmas_df["turma"].tolist()
DIAS = ["Seg", "Ter", "Qua", "Qui", "Sex"]
PERIODOS = [1, 2, 3, 4, 5]
```

3. Arquitetura do Modelo e Restrições (R1 a R7, O1)

As restrições obrigatórias e o objetivo de otimização foram encapsulados na função genérica `construir_modelo`, garantindo a reutilização de código entre os horários gerados:

- **R1 e R5:** Nenhuma turma ou professor pode ter aulas em simultâneo.

$$\forall_{t,d,per} \sum X \leq 1 \quad \text{e} \quad \forall_{p,d,per} \sum X \leq 1$$

- **R2:** Cumprimento da carga semanal exata por disciplina.

$$\forall_{t,disc} \sum X = C_{disc}$$

- **R3 e R4:** No máximo uma ocorrência diária por disciplina, ou blocos estritamente consecutivos para as de `duplo_periodo` (aplicado via `add_allowed_assignments`).
- **R6:** Respeito absoluto pelas indisponibilidades dos docentes reportadas nos dados.

$$\forall_{(p,d,per) \in Exc} \sum X = 0$$

- **R7:** Alocação correta a salas normais ou especiais e respeito pelas capacidades máximas simultâneas de cada sala.

$$\forall_{s,d,per} \sum X \leq Q_s$$

- **O1:** Minimização do total de buracos nos horários dos professores (*amplitude* – *total_aulas*).

```
In [ ]: def construir_modelo(turmas_df, disciplinas_df, salas_df, excecoes_df,
dias, periodos, com_ol=True):
    turmas = turmas_df["turma"].tolist()
    salas = salas_df["sala"].tolist()
    professores = disciplinas_df["professor"].unique().tolist()
```

```

curriculo = []
for _, linha in disciplinas_df.iterrows():
    especial = linha["sala_especial"] if
isinstance(linha["sala_especial"], str) else None
    curriculo.append((linha["disciplina"], linha["professor"],
                        linha["carga_semanal"], linha["duplo_periodo"],
                        especial))

modelo = cp_model.CpModel()

X = {}
for (disc, prof, carga, duplo, especial) in curriculo:
    for t in turmas:
        for d in dias:
            for per in periodos:
                for s in salas:
                    X[(t, disc, prof, d, per, s)] =
modelo.new_bool_var(
                        f"aula_{t}_{disc}_{prof}_{d}_{per}_{s}")

def aulas(turma=None, disc=None, prof=None, dia=None, per=None,
sala=None):
    escolhidas = []
    for (t, di, p, d, pe, s), var in X.items():
        if turma is not None and t != turma: continue
        if disc is not None and di != disc: continue
        if prof is not None and p != prof: continue
        if dia is not None and d != dia: continue
        if per is not None and pe != per: continue
        if sala is not None and s != sala: continue
        escolhidas.append(var)
    return escolhidas

# R1: uma turma não tem duas aulas em simultâneo
for t in turmas:
    for d in dias:
        for per in periodos:
            modelo.add_at_most_one(aulas(turma=t, dia=d, per=per))

# R2: carga semanal exata
for t in turmas:
    for (disc, prof, carga, duplo, especial) in curriculo:
        modelo.add(sum(aulas(turma=t, disc=disc, prof=prof)) == carga)

# R3: no máximo uma aula por dia (duas se for duplo período)
for t in turmas:
    for d in dias:
        for (disc, prof, carga, duplo, especial) in curriculo:
            limite = 1 if duplo == "nao" else 2
            modelo.add(sum(aulas(turma=t, disc=disc, prof=prof, dia=d))
<= limite)

# R4: duplo período = bloco de 2 tempos consecutivos
n = len(periodos)
padroes = [tuple([0] * n)]
for i in range(n - 1):

```

```

        padrao = [0] * n
        padrao[i] = 1
        padrao[i + 1] = 1
        padroes.append(tuple(padrao))

    for (disc, prof, carga, duplo, especial) in curriculo:
        if duplo == "sim":
            for t in turmas:
                for d in dias:
                    presenca = []
                    for per in periodos:
                        var_aux =
modelo.new_bool_var(f"acontece_{t}_{disc}_{d}_{per}")
                        modelo.add(sum(aulas(turma=t, disc=disc, prof=prof,
dia=d, per=per)) == var_aux)
                        presenca.append(var_aux)
                    modelo.add_allowed_assignments(presenca, padroes)

    # R5: um professor não dá duas aulas em simultâneo
    for prof in professores:
        for d in dias:
            for per in periodos:
                modelo.add_at_most_one(aulas(prof=prof, dia=d, per=per))

    # R6: indisponibilidades dos professores
    for _, linha in excecoes_df.iterrows():
        proibidas = aulas(prof=linha["professor"], dia=linha["dia"],
per=linha["periodo"])
        if proibidas:
            modelo.add(sum(proibidas) == 0)

    # R7: tipo de sala certo e capacidade das salas
    for (disc, prof, carga, duplo, especial) in curriculo:
        for _, linha in salas_df.iterrows():
            sala = linha["sala"]
            tipo = linha["tipo"]
            sala_errada = (especial is None and tipo != "normal") or \
                (especial is not None and sala != especial)
            if sala_errada:
                for var in aulas(disc=disc, prof=prof, sala=sala):
                    modelo.add(var == 0)

    for d in dias:
        for per in periodos:
            for _, linha in salas_df.iterrows():
                modelo.add(sum(aulas(dia=d, per=per, sala=linha["sala"]))
<= linha["quantidade"])

    # O1: minimizar os buracos dos professores
    if com_o1:
        buracos_totais = []
        for prof in professores:
            for d in dias:
                tem_aula_no_per = {}
                for per in periodos:
                    tem_aula =

```

```

modelo.new_bool_var(f"trab_{prof}_{d}_{per}")
aulas_per = aulas(prof=prof, dia=d, per=per)
modelo.add(sum(aulas_per) >=
1).only_enforce_if(tem_aula)
modelo.add(sum(aulas_per) ==
0).only_enforce_if(tem_aula.Not())
tem_aula_no_per[per] = tem_aula

primeiro_per = modelo.new_int_var(1, n, f"prim_{prof}_{d}")
ultimo_per = modelo.new_int_var(1, n, f"ult_{prof}_{d}")
tem_aulas_no_d = modelo.new_bool_var(f"tem_dia_{prof}_{d}")

total_aulas = modelo.new_int_var(0, n,
f"total_aulas_{prof}_{d}")
modelo.add(total_aulas == sum(aulas(prof=prof, dia=d)))
modelo.add(total_aulas >=
1).only_enforce_if(tem_aulas_no_d)
modelo.add(total_aulas ==
0).only_enforce_if(tem_aulas_no_d.Not())

for per in periodos:
    modelo.add(primeiro_per <=
per).only_enforce_if(tem_aula_no_per[per])
    modelo.add(ultimo_per >=
per).only_enforce_if(tem_aula_no_per[per])

amplitude = modelo.new_int_var(0, n, f"amp_{prof}_{d}")
buracos_d = modelo.new_int_var(0, n, f"buracos_{prof}_{d}")
modelo.add(amplitude == ultimo_per - primeiro_per +
1).only_enforce_if(tem_aulas_no_d)
modelo.add(amplitude ==
0).only_enforce_if(tem_aulas_no_d.Not())
modelo.add(buracos_d == amplitude -
total_aulas).only_enforce_if(tem_aulas_no_d)
modelo.add(buracos_d ==
0).only_enforce_if(tem_aulas_no_d.Not())
buracos_totais.append(buracos_d)

modelo.minimize(sum(buracos_totais))

return modelo, X

```

```

In [ ]: def extrair_horario(solver, X):
    chaves = [k for k, var in X.items() if solver.Value(var) == 1]
    linhas = []
    for (t, disc, prof, d, per, s) in chaves:
        linhas.append({"Turma": t, "Dia": d, "Período": per,
                        "Detalhe": f"{disc} ({prof}) - [{s}]"})
    return chaves, pd.DataFrame(linhas)

def desenhar_grelhas(horario_df, turmas, dias, sufixo=""):
    elementos = []
    for turma in turmas:
        elementos.append(mo.md(f"### Horário da Turma: {turma}{sufixo}"))
        grelha = horario_df[horario_df["Turma"] == turma].pivot_table(
            index="Período", columns="Dia", values="Detalhe",

```

```

        aggfunc="first", fill_value="--")
    grelha = grelha[[d for d in dias if d in grelha.columns]]
    elementos.append(grelha)
    return mo.vstack(elementos)

```

4. Resolução do Horário Base (H0)

Acionamos o solver CP-SAT para encontrar o horário base inicial otimizando a minimização de buracos dos docentes (*O1*). Guardamos as chaves ativas em `chaves_h0` para servir de âncora à fase seguinte.

```

In [ ]: modelo, X = construir_modelo(turmas_df, disciplinas_df, salas_df,
                                     excocoes_df,
                                     DIAS, PERIODOS, com_ol=True)
print(f"Foram criadas {len(X)} variáveis de decisão!")

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 30.0
solver.parameters.num_search_workers = 1
status = solver.Solve(modelo)

chaves_h0, horario_df = [], None
if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    print(f"Buracos totais: {solver.ObjectiveValue()} | tempo:
    {solver.WallTime():.2f}s | {solver.StatusName(status)}")
    chaves_h0, horario_df = extrair_horario(solver, X)
    print(f"Foram guardadas {len(chaves_h0)} aulas do H0!")
    saida_h0 = desenhar_grelhas(horario_df, TURMAS, DIAS)
else:
    saida_h0 = mo.md(f"Sem solução: {solver.StatusName(status)}")

saida_h0

```

5. Construção Incremental e Reescalonamento (H1 / R9)

Para o horário H1 (com os novos `dados_v2`), a estratégia adotada baseia-se na restrição *R9*: reaproveitar a estrutura anterior como ponto de partida (*hint*) e maximizar as aulas que se mantêm inalteradas, garantindo um reescalonamento cirúrgico e eficiente.

Verificou-se que para a alteração introduzida em `dados_v2` (nova indisponibilidade da Prof. Ana), o horário base *H0* já respeitava a restrição, resultando em 0 aulas alteradas e num tempo de reescalonamento inferior a 0.05 segundos. Caso o modelo fosse resolvido inteiramente do zero, o solver descartaria a estrutura anterior e exigiria uma nova busca exaustiva, comprovando-se assim a vantagem da otimização por re-ancoragem.

```

In [ ]: turmas2_df = pd.read_csv("dados_v2/turmas.csv")
disciplinas2_df = pd.read_csv("dados_v2/disciplinas.csv")
salas2_df = pd.read_csv("dados_v2/salas.csv")
excecoes2_df = pd.read_csv("dados_v2/disponibilidade_excecoes.csv")
TURMAS2 = turmas2_df["turma"].tolist()

modelo2, X2 = construir_modelo(turmas2_df, disciplinas2_df, salas2_df,
                               excecoes2_df,
                               DIAS, PERIODOS, com_ol=False)

# R9: reaproveitar o H0 (hint) e maximizar as aulas que ficam iguais
_set_h0 = set(chaves_h0)
for _chave, _var in X2.items():
    modelo2.add_hint(_var, 1 if _chave in _set_h0 else 0)
modelo2.maximize(sum(_var for _chave, _var in X2.items() if _chave in
_set_h0))

solver2 = cp_model.CpSolver()
solver2.parameters.max_time_in_seconds = 30.0
solver2.parameters.num_search_workers = 1
status2 = solver2.Solve(modelo2)
print(f"Estado do solver (H1): {solver2.StatusName(status2)} | tempo:
{solver2.WallTime():.3f}s")

horario_h1_df = None
if status2 in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    chaves_h1, horario_h1_df = extrair_horario(solver2, X2)
    aulas_mantidas = len(_set_h0 & set(chaves_h1))
    aulas_alteradas = len(chaves_h0) - aulas_mantidas
    saida_h1 = mo.vstack([
        mo.md("### Estatísticas do Reescalonamento (H1)",
        mo.md(f"***Aulas mantidas sem alteração:** {aulas_mantidas}"),
        mo.md(f"***Aulas que tiveram de mudar:** {aulas_alteradas}"),
        desenhar_grelhas(horario_h1_df, TURMAS2, DIAS, " (H1)"),
    ])
else:
    saida_h1 = mo.md("0 solver não conseguiu encontrar um horário válido
para os novos dados.")

saida_h1

```

6. Cenário Inédito (dados_v3)

Para testar a robustez e provar a ausência de elementos fixos no código (*hardcoding*), aplicamos o nosso modelo modular a um novo conjunto de dados completo presente em `dados_v3`, validando a capacidade de generalização do algoritmo perante novas turmas, professores e espaços escolares.

```

In [ ]: turmas3_df = pd.read_csv("dados_v3/turmas.csv")
disciplinas3_df = pd.read_csv("dados_v3/disciplinas.csv")
salas3_df = pd.read_csv("dados_v3/salas.csv")
excecoes3_df = pd.read_csv("dados_v3/disponibilidade_excecoes.csv")

```

```

# Limpeza de espaços em branco (defesa contra erros de formatação nos CSVs)
for _df in [turmas3_df, disciplinas3_df, salas3_df, excecoes3_df]:
    for _col in _df.select_dtypes(include=['object']).columns:
        _df[_col] = _df[_col].str.strip()

TURMAS3 = turmas3_df["turma"].tolist()

modelo3, X3 = construir_modelo(turmas3_df, disciplinas3_df, salas3_df,
                               excecoes3_df,
                               DIAS, PERIODOS, com_ol=True)

solver3 = cp_model.CpSolver()
solver3.parameters.max_time_in_seconds = 30.0
solver3.parameters.num_search_workers = 1
status3 = solver3.Solve(modelo3)
print(f"V3: {solver3.StatusName(status3)} | tempo: {solver3.WallTime():.2f}s")

horario_v3_df = None
if status3 in (cp_model.OPTIMAL, cp_model.FEASIBLE):
    print(f"Buracos totais: {solver3.ObjectiveValue()}")
    _, horario_v3_df = extrair_horario(solver3, X3)
    saida_v3 = desenhar_grelhas(horario_v3_df, TURMAS3, DIAS, " (V3)")
else:
    saida_v3 = mo.md("Sem solução para o cenário V3.")

saida_v3

```

7. Verificação Automática do Modelo

Apresentamos um auditor dinâmico em Pandas que valida de forma cega o cumprimento rigoroso das restrições **R1 a R7** para todos os horários gerados (*H0*, *H1* e *V3*). Para demonstrar a eficácia da detecção de falhas desta ferramenta, incluímos um **Teste Negativo** alimentado com um horário corrompido propositalmente através da duplicação manual de instâncias.

```

In [ ]: def validar_horario(nome_horario, df_horario, df_t, df_d, df_s, df_e,
                             lista_turmas):
    if df_horario is None or df_horario.empty:
        return mo.md(f"### Validação {nome_horario}: Sem dados para validar.")

    _df_teste = df_horario.copy()
    _df_teste['Disciplina'] = _df_teste['Detalhe'].str.split(r' \(').str[0]
    _df_teste['Professor'] = _df_teste['Detalhe'].str.extract(r'\((.*?)\)')
    _df_teste['Sala'] = _df_teste['Detalhe'].str.extract(r'\[(.*?)\]')

    erros = 0
    resultados = [f"### Verificação Automática: {nome_horario}"]

    # R1: Sobreposições de turmas
    _sobrepos_t = _df_teste.groupby(['Turma', 'Dia', 'Período']).size()
    if not _sobrepos_t[_sobrepos_t > 1].empty:

```

```

        resultados.append("**Falha no R1:** Turmas com aulas sobrepostas.")
        erros += 1
    else:
        resultados.append("**R1 OK**")

    # R2: Carga Semanal
    _erro_r2 = False
    for _t in lista_turmas:
        for _, _row in df_d.iterrows():
            _esperado = _row['carga_semanal']
            _marcado = len(_df_teste[( _df_teste['Turma'] == _t) &
            (_df_teste['Disciplina'] == _row['disciplina'])])
            if _marcado != _esperado:
                _erro_r2 = True
        resultados.append("**Falha no R2**" if _erro_r2 else "**R2 OK**")
    if _erro_r2: erros += 1

    # R3 e R4: Ocorrências e Blocos
    _erro_r3, _erro_r4 = False, False
    for _t in lista_turmas:
        for _, _row in df_d.iterrows():
            _aulas = _df_teste[( _df_teste['Turma'] == _t) &
            (_df_teste['Disciplina'] == _row['disciplina'])]
            for _dia, _periodos in _aulas.groupby('Dia')
            ['Período'].apply(list).items():
                if _row['duplo_período'] == 'nao' and len(_periodos) > 1:
                    _erro_r3 = True
                elif _row['duplo_período'] == 'sim':
                    if len(_periodos) > 2: _erro_r3 = True
                    if len(_periodos) == 2 and abs(_periodos[1] -
                    _periodos[0]) != 1: _erro_r4 = True
                    elif len(_periodos) == 1: _erro_r4 = True
        resultados.append("**Falha no R3**" if _erro_r3 else "**R3 OK**")
        resultados.append("**Falha no R4**" if _erro_r4 else "**R4 OK**")
    if _erro_r3: erros += 1
    if _erro_r4: erros += 1

    # R5: Sobreposições Professor
    _sobrepos_p = _df_teste.groupby(['Professor', 'Dia', 'Período']).size()
    if not _sobrepos_p[_sobrepos_p > 1].empty:
        resultados.append("**Falha no R5:** Professores com aulas
sobrepostas.")
        erros += 1
    else:
        resultados.append("**R5 OK**")

    # R6: Exceções
    _v_r6 = pd.merge(_df_teste, df_e.rename(columns={'professor':
'Professor', 'dia': 'Dia', 'periodo': 'Período'}), on=['Professor', 'Dia',
'Período'])
    if not _v_r6.empty:
        resultados.append("**Falha no R6:** Aulas marcadas em
indisponibilidades.")
        erros += 1
    else:
        resultados.append("**R6 OK**")

```

```

# R7: Salas
_erro_r7 = False
_ocupacao = pd.merge(_df_teste.groupby(['Sala', 'Dia',
'Período']).size().reset_index(name='Ocupacao'), df_s.rename(columns=
{'sala': 'Sala'}), on='Sala')
if not _ocupacao[_ocupacao['Ocupacao'] >
_ocupacao['quantidade']].empty: _erro_r7 = True

for _, row in _df_teste.iterrows():
    _req = str(df_d[df_d['disciplina'] == row['Disciplina']]
['sala_especial'].values[0])
    _tipo = df_s[df_s['sala'] == row['Sala']]['tipo'].values[0]
    if (_req == 'nan' and _tipo != 'normal') or (_req != 'nan' and
row['Sala'] != _req): _erro_r7 = True

resultados.append("**Falha no R7:** Lotação ou tipo violado." if
_erro_r7 else "**R7 OK**")
if _erro_r7: erros += 1

resultados.append("\n**Sucesso!**" if erros == 0 else f"\n**{erros}
erros!**")
return mo.vstack([mo.md(linha) for linha in resultados])

saida_validar_h0 = validar_horario("Horário Base (H0)", horario_df,
turmas_df, disciplinas_df, salas_df, excecoes_df, TURMAS)
saida_validar_h1 = validar_horario("Horário Incremental (H1)",
horario_h1_df, turmas2_df, disciplinas2_df, salas2_df, excecoes2_df,
TURMAS2)
saida_validar_v3 = validar_horario("Cenário Inédito (V3)", horario_v3_df,
turmas3_df, disciplinas3_df, salas3_df, excecoes3_df, TURMAS3)

horario_estragado = pd.concat([horario_df.copy(), horario_df.iloc[[0]],
ignore_index=True)
saida_estragada = validar_horario("Horário Estragado (Teste Negativo)",
horario_estragado, turmas_df, disciplinas_df, salas_df, excecoes_df,
TURMAS)

mo.vstack([saida_validar_h0, mo.md("---"), saida_validar_h1, mo.md("---"),
saida_validar_v3, mo.md("---"), saida_estragada])

```

8. Conclusão e Limitações do Modelo

O presente trabalho prático demonstrou com sucesso a aplicação de Programação por Restrições (através do solver CP-SAT) à resolução de um problema clássico e denso de escalonamento: a criação de horários escolares.

Principais Resultados Alcançados:

- **Robustez e Dinamismo:** O modelo matemático implementado respeita estritamente todas as restrições obrigatórias (R1 a R8). Ao extrair os dados dinamicamente de ficheiros CSV, o código garante que não existem variáveis fixas

(*hardcoded*), adaptando-se a qualquer currículo, lotação de salas ou indisponibilidade.

- **Reescalonamento Eficiente (R9):** A abordagem de construção incremental revelou-se altamente eficaz. Ao injetar o horário base ($H0$) como ponto de partida (*hint*) e maximizar a intersecção de aulas mantidas, o modelo consegue acomodar imprevistos de recursos de forma cirúrgica, evitando a reestruturação desnecessária de todo o ecossistema escolar.
- **Auditoria Automática:** A construção de um validador programático em Pandas garante a fiabilidade e integridade dos resultados, provando de forma transparente e automatizada que nenhuma restrição lógica foi violada.

Limitações e Escalabilidade: Apesar da grande eficácia do modelo, a componente de otimização ($O1$ - minimização de "buracos" no horário dos docentes) acarreta um custo computacional significativo. O cálculo da amplitude de trabalho obriga à criação de múltiplas variáveis de controlo auxiliares ($5 \times D \times Prof$). Embora o solver resolva os cenários de teste ($H0, H1, V3$) em frações de segundo, a extrapolação deste objetivo estrito para uma escola real com centenas de turmas e professores levaria a uma explosão combinatória. Em ambientes de escala massiva, seria necessário impor limites de tempo de pesquisa rigorosos ou adotar uma abordagem heurística relaxada para a otimização.

Em suma, a solução desenvolvida cumpre com rigor os requisitos exigidos, apresentando-se como uma ferramenta de Lógica Computacional modular, elegante e altamente adaptável.

9. Declaração de utilização de LLM

Na elaboração deste projeto utilizei o assistente de IA CLaude (Anthropic) com os seguintes objetivos:

1. **Verificação do trabalho realizado**, em particular a revisão das restrições do modelo e a identificação de possíveis erros.
2. **Refatoração do código**, com a ajuda da criação de funções reutilizáveis (`construir_modelo` , `extrair_horario` e `desenhar_grelhas`) para evitar a repetição de código nos cenários $H0, H1$ e $V3$.
3. **Orientação para a integração** dessas funções no notebook, incluindo a ordem das alterações.